

Columbia University
Department of Electrical Engineering
EE E4321. Final project.
Fall, 2023

In this project, you will design a *very simple* microcontroller core. We will assemble the components of this core as part of the remaining problem sets (as we cover in class the details of dataflow functional unit design, register files and memories, and control logic design). In the final week or two of the course, you will then assemble the pieces of your design (as we discuss in class global issues such as floorplanning, global wiring, and clock and power distribution). You will complete all aspects of the design project working in groups of two. Please try to find a partner as soon as possible and see me if you are having any difficulty.

By the final due date for the project, you will have a completed layout that is LVS and DRC clean. The design will use a combination of Spectre and Ultrasim simulations to verify the clock cycle time, to calculate power dissipation, and to verify that the core is functionally correct.

We are designing an 8-bit accumulator-based core with a 6-bit instruction word and 3-bit opcode. The core has the following interface to the external world:

Signal	Direction	Description
ϕ_1	Input	Phase 1 clock
ϕ_2	Input	Phase 2 clock
$instr < 0 : 5 >$	Input	Instruction to execute
$bus < 0 : 7 >$	Bidi	Processor-memory bus
$load_{bus}$	Output	Load the internal bus externally
$store_{bus}$	Output	Load the internal bus to the external bus

The opcode, which is $instr < 0 : 2 >$, is encoded as shown below. Where appropriate, the memory address is encoded in $instr < 3 : 5 >$. For the shift operation, $instr < 3 : 5 >$, instead codes the shift amount.

The datapath for this machine is shown in Figure 1. Note that each instruction takes exactly one cycle to complete so that a new instruction can (and will) be presented to the core during each cycle. We will discuss the design of each of these components in subsequent problem sets.

Opcode	Assembly	Description
000	NOP	Do nothing
001	LOAD	$\text{Mem}[i] \leftarrow \text{External bus}$
010	STORE	$\text{External bus} \leftarrow \text{Mem}[i]$
011	GET	$\text{Acc} \leftarrow \text{Mem}[i]$
100	PUT	$\text{Mem}[i] \leftarrow \text{Acc}$
101	ADD	$\text{Acc} \leftarrow \text{Acc} + \text{Mem}[i]$
110	SUB	$\text{Acc} \leftarrow \text{Acc} - \text{Mem}[i]$
111	SHIFT	Left logical shift of Acc

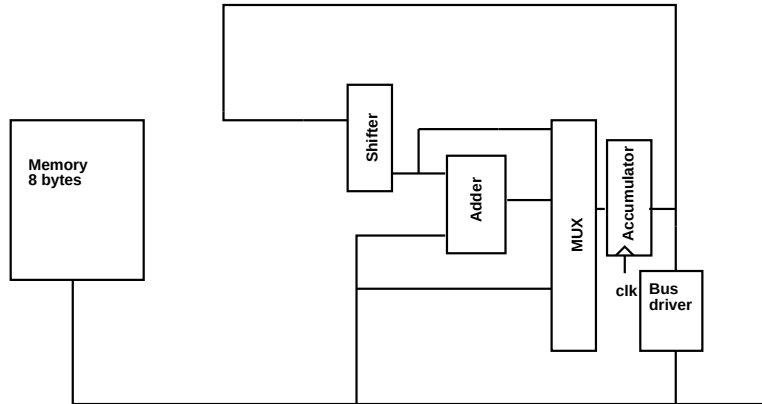


Figure 1: Datapath of the core.

A suggested floorplan for the overall design is shown in Figure 2. Dataflow signals run horizontally and control comes in vertically. The “decoder” decodes the instruction into the control signals needed for each of the dataflow elements. The chip image will have M1 vertical and M2 horizontal. M3 will be reserved for power-ground distribution. As we design these components in subsequent weeks, you should strive to achieve a uniform height for the datapath elements. Take full advantage of the bitslice design style.

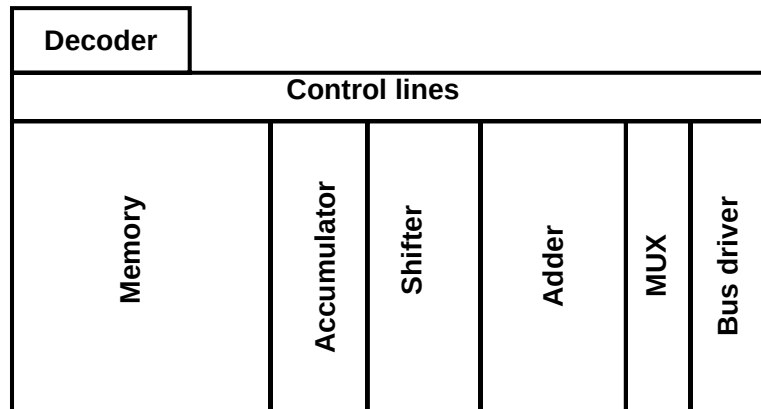


Figure 2: Suggested floorplan of the core.

Your adder should generate an overflow signal. You can just bring it off as an output for the entire design. Normally, this signal would be latched in some kind of condition code which would be available to software to indicate that an overflow has occurred.

You will need to buffer *phi1* and *phi2* internally since they are fairly heavily loaded. (This will be even more true if you do a dynamic design.) Since wire lengths in your design are relatively short, you do not have to be concerned a great deal with RC delays. The challenge is to do a buffer tree for your clock that balances the delays as much as possible – trying to drive equivalent loads from each stage of the buffer tree (with identically sized drivers) is one approach to take.